

NUS Timetable App

A timetable coordination app for NUS students. Users can view their own timetable and compare it with a group to find free slots.

Table of Contents

- [Tech Stack](#)
- [Project Structure](#)
- [Quick Setup](#)
- [Proof of Concept and Getting Started](#)
- [Detailed Page Guide](#)
- [What Has Been Done](#)
- [Roadmap and TODO](#)
- [Development Plan](#)
- [Additional Details](#)

Tech Stack

- **Frontend:** React + Vite
- **Backend:** Python Flask
- **Database:** SQLite

Project Structure

```
orbital/
├── backend/
│   ├── app.py           # Flask app with routes
│   ├── database.py      # SQLite initialization and helpers
│   └── timetable.db      # SQLite database (auto-created)
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── data/
│       │   └── dummyData.js
│       └── components/
```

Roadmap and TODO

- Interactive timetable (real-time availability overlays)
- Integrate NUSMods API for official module timetables
- Group creation and membership management (invite/leave)
- In-app messaging / text capabilities between group members
- Push notifications for scheduling updates and invites
- Improve accessibility and mobile layout
- Add automated tests and CI for backend and frontend

```
## Quick Setup
### Backend
Install Python dependencies and run the backend:
```bash
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
Initialize the database (created automatically on first run)
python app.py
```

## Frontend (development dependencies)

Install Node dependencies; running the dev server is optional for development:

```
cd frontend
npm install
To start the dev server for local development (optional):
npm run dev
```

## Production build (optional)

To build a production-ready frontend bundle:

```
cd frontend
npm run build
```

## Proof of Concept and Getting Started

The minimal proof-of-concept steps are the same as the **Setup** instructions above: start the backend, install frontend deps, and optionally run the Vite dev server to preview the app locally.

## Detailed Page Guide

Below is a detailed explanation of the main files and pages in the project (migrated from `setupExplain.md`). Use this as a reference when editing or extending the app.

### Project Root

- `README.md`: short overview of the app, tech stack, and basic run steps.
- `SETUP.md`: detailed install/run instructions and API notes.
- `.gitignore`: tells Git which generated or local files to ignore.
- `.DS_Store`: macOS Finder metadata, not app code.
- `.git/`: Git's internal metadata folder. Files like `HEAD`, `config`, `index`, `logs`, `refs`, `objects`, and `hooks` are all part of Git history/state, not the app.

### Backend

- `backend/app.py`: Flask API server and route handlers like `/timetable`, `/group`, and `/health`.
- `backend/database.py`: SQLite setup, seed data, and database helper functions.

- `backend/requirements.txt`: Python dependencies Flask needs.
- `backend/timetable.db`: local SQLite database file created by the backend; it stores mock/demo data.
- `backend/__pycache__`/: compiled Python cache files like `.pyc`; safe to delete, Python will recreate them.

## Frontend

- `frontend/index.html`: the HTML entry page that mounts the React app.
- `frontend/package.json`: frontend project config, scripts, and npm dependencies.
- `frontend/package-lock.json`: exact locked npm dependency versions.
- `frontend/vite.config.js`: Vite setup, including dev server port and API proxy to Flask.
- `frontend/dist`/: production build output from `npm run build`.
- `frontend/node_modules`/: installed npm packages and tools. This includes third-party code like React, Vite, Babel, Rollup, and build caches; it is generated by `npm install`.

## Frontend Source

- `frontend/src/main.jsx`: React entry point; renders `<App />` into the page.
- `frontend/src/App.jsx`: main app logic, state, login handling, and switching between personal/group views.
- `frontend/src/App.css`: styles for the main app shell.
- `frontend/src/LoginScreen.jsx`: login/user picker screen.
- `frontend/src/LoginScreen.css`: styles for the login screen.
- `frontend/src/components/GroupView.jsx`: group timetable overlay view, member toggles, and fetching group data.
- `frontend/src/components/GroupView.css`: styles for the group view.
- `frontend/src/components/OverlayGrid.jsx`: renders the color-coded group availability grid and tooltip.
- `frontend/src/components/OverlayGrid.css`: styles for the overlay grid.
- `frontend/src/components/SlotBlock.jsx`: visual block for one timetable slot in the personal grid.
- `frontend/src/components/SlotBlock.css`: styles for slot blocks.
- `frontend/src/components/TimetableGrid.jsx`: personal timetable layout that places slots by day/time.
- `frontend/src/components/TimetableGrid.css`: styles for the timetable grid.

- `frontend/src/data/dummyData.js`: mock module/user/timetable/group data used by the demo.
- `frontend/src/utils/computeOverlay.js`: computes the group availability overlay from member schedules.

### Generated Frontend Build Files

- `frontend/dist/index.html`: generated production HTML entry.
- `frontend/dist/assets/index-22ffc796.js`: bundled production JavaScript.
- `frontend/dist/assets/index-d71316c6.css`: bundled production CSS.

### Hidden/Generated Notes

- `.DS_Store`: macOS Finder metadata, same idea as the root `.DS_Store`.
- `.bin`: command shims for packages like `vite`, `rollup`, and `esbuild`.
- `.vite`: Vite's cache for faster dev reloads.
- `.package-lock.json`: internal npm metadata inside `node_modules`.
- `@babel`, `react`, `vite`, `rollup`, etc.: installed third-party packages, not the app's source.

A short "what to edit for what" cheat sheet can be created to map responsibilities, for example: backend logic, UI components, styling, and generated files.

### Where the real code lives

- Frontend source code (edit for UI/UX and app behavior): `frontend/src/` — React components, styles, mock data, and utilities.
- Frontend build/config: `frontend/vite.config.js`, `frontend/package.json` — edit scripts, ports, and dependency settings.
- Backend source code (edit for API/DB logic): `backend/app.py` and `backend/database.py` — these contain the Flask routes and SQLite schema/queries.
- Backend config/deps: `backend/requirements.txt` — Python packages to install in the `virtualenv`.

Notes:

- Ignore `frontend/node_modules/`, `frontend/dist/`, and `backend/__pycache__/` — they are generated and not for manual edits.
- The runtime database file `backend/timetable.db` contains seeded demo data; to reset it delete the file and restart the backend.

## What Has Been Done

The project currently implements the following features.

Backend (APIs & persistence):

- Session-based demo login/logout (`/login`, `/logout`, `/me`).
- User registration and account reset (`/register`).
- List available users for the login screen (`/users`).
- Timetable retrieval and update endpoints (`/timetable`, `/timetable/update`) which store NUSMods selections and custom events in SQLite (`backend/timetable.db`).
- Group creation and query endpoints (`/group` POST, `/group/<id>` GET) including returning all members' timetable data.
- Health and root endpoints (`/health`, `/`) for simple checks and endpoint listing.
- Database schema and seeding for users, timetable entries, custom events, groups, and group\_members in `backend/database.py` (includes safe pruning of legacy demo accounts).

#### Frontend (UI & behaviour):

- Login screen showing demo accounts and ability to create a new blank account (`frontend/src/LoginScreen.jsx`).
- Personal timetable view that renders NUSMods module lessons and custom events (`frontend/src/components/TimetableGrid.jsx`, `SlotBlock.jsx`).
- Add custom events UI (title, day, start/end, color) and delete via a modal — changes are stored in local state and synced by the app to the backend where supported (`frontend/src/App.jsx`).
- Group view overlay that fetches a group's members and displays a colour-coded availability grid; toggle members on/off to change the overlay (`frontend/src/components/GroupView.jsx`, `OverlayGrid.jsx`).
- Overlay computation algorithm that aggregates member busy counts into traffic-light colouring and supplies per-block busy member lists (`frontend/src/utils/computeOverlay.js`).
- Merge logic to combine NUSMods selections with custom events into unified slot objects used by the UI.

#### Other notable pieces:

- Mock NUSMods module data and demo users shipped in `frontend/src/data/dummyData.js` for offline/demo use.
- Vite development proxy configured to forward API calls to the Flask backend (see `frontend/vite.config.js`).
- CSS components and layout for timetable, overlay and login screens (files under `frontend/src/components/*.css`).
- Development convenience features: DB seeding with sample users/groups, automatic sample-group membership for new accounts, and small backend protections to avoid re-

creating legacy placeholder accounts.

## Additional Details

- Backend runs at `http://127.0.0.1:5001` by default; frontend dev server (Vite) serves at `http://localhost:3000` when started.
- Vite proxies API calls to the backend (see `frontend/vite.config.js`).
- To reset seeded data delete `backend/timetable.db` and restart the backend (it will re-seed as configured).
- See `SETUP.md` for extra installation notes and API specifics. |     |  
TimetableGrid.jsx |     | — GroupView.jsx |     | — SlotBlock.jsx

### ## Development Plan

This development plan lists the targeted work to complete by 2026-06-30 and proposed features for July 2026. Assignments assume a mix of human engineers and AI-assisted scaffolding; adjust owners and dates as needed.

### ### June Targets (complete by 2026-06-30)

- Branch & PR — due 2026-06-01
  - Create a feature branch for README and docs edits, commit locally, open a PR for review. Owner: Developer (AI: PR description + checklist).
- Backend stability & monitoring — due 2026-06-05
  - Investigate cause of recent ``exit 137``, add logging, and provide a supervisor/runbook (nohup/systemd/gunicorn) recommendation. Owner: Ops/Dev.
- Timetable UX polish — due 2026-06-07
  - Finalize ``frontend/src/components/TimetableGrid.jsx`` spacing/flex fixes and verify across browsers. Owner: Frontend.
- DB hygiene & scripts — due 2026-06-10
  - Add a small SQL cleanup script and document reset/seed steps (``backend/database.py`` already contains pruning). Owner: Backend.
- NUSMods integration (MVP) — due 2026-06-20
  - Implement a simple importer: fetch NUSMods JSON, normalize lesson entries, and save ``moduleSelections``. Owner: Backend + Frontend.
- Interactive timetable improvements (MVP) — due 2026-06-25
  - Improve overlay responsiveness, member toggles, and live edits reflected in ``GroupView``. Owner: Frontend.
- Group creation & UI flows (MVP) — due 2026-06-25
  - Add a minimal create-group form and member picker that POSTs to ``/group``. Owner: Frontend.
- Tests & CI (basic) — due 2026-06-28
  - Add backend unit tests for DB helpers and a frontend smoke test; configure GitHub Actions to run them. Owner: Dev + AI test scaffolding.
- Accessibility & mobile fixes — due 2026-06-29
  - Address keyboard navigation, ARIA labels, and responsive layout for main pages. Owner: Frontend.
- Docs, changelog, June release notes — due 2026-06-30
  - Finalize ``README.md``, ``SETUP.md``, and a short changelog summarizing June work. Owner: Docs/Dev.

### ### July Proposed Features (consider for July sprints)

- In-app messaging (chat MVP) — proposed 2026-07-01 → 2026-07-07
  - Lightweight chat stored in SQLite with basic UI; scope for MVP only. Owner: Backend + Frontend.
- Push notifications (web push) — proposed 2026-07-08 → 2026-07-14
  - Research VAPID keys and implement minimal push subscription + demo notification flows. Owner: Dev/Ops.
- Module search & bulk import UI — proposed 2026-07-10 → 2026-07-17

- Add a searchable NUSMods lookup and a bulk import dialog for module selections. Owner: Frontend.
- Invitation & email notifications – proposed 2026-07-15 → 2026-07-21
  - Invite users to groups via email (SMTP or transactional API). Owner: Backend.
- End-to-end tests (Playwright) – proposed 2026-07-18 → 2026-07-24
  - Add E2E tests covering login, timetable rendering, group overlay interactions. Owner: QA/Dev.
- Polish & release candidate – proposed 2026-07-25 → 2026-07-31
  - Final UI polish, performance checks, and prepare release notes. Owner: All.

### Sprint cadence & process notes

- Sprint length: 1 week (Mon planning → Fri demo). Keep tasks small (1-3 days).